# STRATEGIC FRAMEWORK FOR PREDICTIVE MAINTENANCE

## Engineering Reliability, Data Analytics, and Financial Lifecycle Valuation of Industrial Assets

By Cajethan Nwangwu 16/04/2026

**Table of content:**


- 1. Introduction
- 2. Ask Phase
  - 2.1 Establishing the Strategic and Thermodynamic Context
  - 2.2 Objective
  - 2.3 Economic and Strategic Imperatives for Industrial Assets
- 3. Prepare Phase
  - 3.1 Data Integrity and Multi-Variate Sensor Analysis
  - 3.2 The C-MAPSS Data Architecture
  - 3.3 Programmatic Download and Extraction
- 4. Process Phase
  - 4.1 Feature Engineering and Temporal Context
  - 4.2 Engineering the Time-to-Failure (TTF) Label
  - 4.3 Rolling Statistics with the {slider} Package
- 5. Analyze Phase
  - 5.1 Sensor Correlation and Machine Learning Optimization
  - 5.2 Validating Predictive Power with PerformanceAnalytics
  - 5.3 The tidymodels Optimization Framework
  - 5.4 Visualization
- 6. Conclusion


### 1. Introduction:

The integration of advanced data analytics into the maintenance of critical industrial infrastructure marks a fundamental shift from traditional reactive models to a proactive, predictive paradigm. In the context of high-value assets such as turbofan engines, compressors, and large-scale pumps found in refineries or petrochemical plants, the failure of a single component is not merely a technical glitch but a systemic financial and operational crisis. This project examines the implementation of a **predictive maintenance (PdM**) framework using the **NASA Turbofan Engine Degradation Simulation Dataset (C-MAPSS)**, synthesized through the multi-disciplinary lenses of chemical engineering, and data analytics.

The primary challenge in modern industrial environments involves managing the escalating cost of unplanned downtime, which industry benchmarks suggest can reach between $200,000 and over $500,000 per hour in the oil and gas sector. In manufacturing firms, this cumulative impact amounts to an estimated $1.4 trillion annually, representing approximately 11% of total global revenue. Moving beyond "Run-to-Failure" strategies requires a robust analytical process that identifies early signals of degradation—the "Potential Failure" point on the PF curve—to allow for planned interventions that minimize functional failure and secondary system damage.

## 2. Ask Phase

### 2.1 Establishing the Strategic and Thermodynamic Context:

In this phase of the analytics process, my objective is to align maintenance activities with overarching business goals, specifically the optimization of **Asset Residual Value** and the maximization of **Overall Plant Reliability (OPR)**. In the petrochemical industry, OPR is a holistic metric that reflects the combined impact of asset availability, process speed, and production quality across multiple units. This differs from **Overall Equipment Effectiveness (OEE)**, which focuses on individual asset performance.

From a chemical engineering perspective, the turbofan engine—a complex assembly of compressors, burners, and turbines—operates on the Brayton cycle. Its health is dictated by thermodynamic efficiencies at various stages, such as the High-Pressure Compressor (HPC) and the Low-Pressure Turbine (LPT). Degradation, such as fouling or erosion of turbine blades, manifests as subtle anomalies in pressure ratios and exhaust temperatures.

### 2.2 Objective:

To build a classification model that predicts whether a specific unit is at "High Risk" of failure within the next 30 days, thereby allowing for the transition from a calendar-based preventive schedule to a condition-based predictive program.

### 2.3 Economic and Strategic Imperatives for Industrial Assets:

Asset managers view industrial equipment as long-term capital investments whose value is derived from their ability to generate predictable Net Operating Income (NOI). The "replacement asset value" (RAV) of a large-scale refinery unit can reach hundreds of millions of dollars, and research reveals that maintenance costs typically range from 2% to 4% of this total value annually. When maintenance is reactive, costs escalate significantly due to emergency labor premiums (often 1.5x to 2x standard rates) and expedited shipping for critical spares.

The decision to adopt a PdM framework is driven by the "40x cost difference" documented in comparative analyses of emergency versus planned repairs (ROI Guide for Industrial Plants). A planned repair of a seal or bearing might cost CAD $6,500, whereas the same repair performed as an emergency response following a functional failure could cost upwards of CAD $261,000 when production losses are factored in.

## 3. The Prepare Phase

### 3.1 Data Integrity and Multi-Variate Sensor Analysis:

I made use of the NASA C-MAPSS dataset, which offers run-to-failure trajectories for multiple aircraft engines under realistic flight conditions. The dataset represents a simulation framework where engines start with varying degrees of initial wear and manufacturing variation—noise that is considered "normal" but must be accounted for in the predictive model.

### 3.2 The C-MAPSS Data Architecture:

The dataset includes 21 sensors capturing multi-variate health parameters. For a chemical engineer, these sensors are the "vital signs" of the thermodynamic process.

In the R environment, preparing this data requires careful structural management to ensure that unit IDs and cycles are treated as temporal identifiers rather than predictors.

**R code chunk**

```{r installing packages}

install.packages("tidyverse")  # Data wrangling and visualization

install.packages("tidymodels") # Reproducible modeling framework

install.packages("slider")     # Rolling window statistics for degradation trends

install.packages("degradr")

install.packages("here")

install.packages("PerformanceAnalytics") #Multi-variate sensor correlation

install.packages("ranger")

install.packages("rlang")

install.packages("rlang", repos = "https://cran.rstudio.com/")
```

```{r loading packages}

library(tidyverse)

library(here)

library(rsample)

library(recipes)

library(parsnip)

library(nlme)

library(slider)

library(PerformanceAnalytics)
```

**3.3 Programmatic Download and Extraction**

To downloads the official NASA C-MAPSS zip file and extracts the specific training file required for the analysis, I ran the code below.

```{r NASA file extraction}
```

```
# Define the URL and destination file name
dataset_url <- "https://data.nasa.gov/docs/legacy/CMAPSSData.zip"
zip_file <- "CMAPSSData.zip"
target_file <- "train_FD001.txt"

# Download the dataset if it doesn't already exist
if (!file.exists(zip_file)) {
  download.file(dataset_url, destfile = zip_file, mode = "wb")
}

# Unzip the contents into the current directory
unzip(zip_file, overwrite = TRUE)

# Verify the file now exists
if (file.exists("train_FD001.txt")) {
  print("File successfully acquired.")
} else {
  stop("File download or extraction failed.")
}
```

```{r defining schema}
# Defining NASA C-MAPSS Schema (26 columns, no headers in raw file)
col_names <- c("unit_id", "cycle", "setting_1", "setting_2", "setting_3", paste0("s_", 1:21))

# Numeric-Safe Loading: Forces numeric types to bypass GTM/JavaScript strings
train_raw <- read_delim(target_file, delim = " ", col_names = col_names,
                        trim_ws = TRUE, col_types = cols(.default = col_double())) %>%
  select(where(~!all(is.na(.)))) # Remove phantom trailing columns common in NASA files
```

The data preparation stage must also acknowledge the "PF interval"—the time between potential failure detection and functional failure. In the C-MAPSS dataset, this interval is represented by the trend of sensor values as they deviate from their initial healthy baseline toward the point of catastrophic failure at the end of the time series.

**4. The Process Phase**

**4.1 Feature Engineering and Temporal Context:**

The "Process" phase is the most critical for an engineer because it transforms raw sensor readings into predictive features that capture the velocity and acceleration of degradation. Standard raw readings often contain noise from operational fluctuations; therefore, the "Engineering Edge" is achieved by creating lagging variables and rolling statistics that smooth the signal and highlight the trend.

**4.2 Engineering the Time-to-Failure (TTF) Label:**

The primary target for the model is the Remaining Useful Life (RUL), or Time-to-Failure. In the training set, every unit is followed until failure, allowing us to calculate the RUL as the difference between the maximum cycle recorded for a unit and its current cycle.

$$\mathrm{RUL}_{i,t} = \max(\mathrm{cycle}_i) - \mathrm{cycle}_{i,t}$$

From an asset valuation perspective, RUL is a proxy for the remaining capital value of the equipment. A unit nearing the end of its RUL represents a significant liability, as it necessitates immediate CAPEX for replacement or overhaul.

**4.3 Rolling Statistics with the {slider} Package:**

Using the slider package, we generate rolling averages over the last 5 cycles for critical sensors like s_11 (Ps30) and s_15 (Bypass Ratio). These rolling averages act as a "low-pass filter," removing high-frequency noise and focusing the model on the underlying degradation path.

```{r the engineering edge}
train_engineered <- train_raw %>%
  group_by(unit_id) %>%
  mutate(
    # Engineering Time-to-Failure (TTF) / Remaining Useful Life (RUL)
    rul = max(cycle) - cycle,

    # Classification Label: High Risk within 30 cycles (approx 30 days)
```

```
  target_label = factor(if_else(rul <= 30, "HighRisk", "Normal"),
                        levels = c("HighRisk", "Normal")),

  # Engineering Edge: 5-cycle rolling averages to smooth sensor noise
  across(starts_with("s_"),
         ~slide_dbl(.x, mean,.before = 4,.complete = TRUE),
         .names = "{.col}_roll_avg")
 ) %>%
 ungroup() %>%
 drop_na() %>% # Removes rows where rolling windows are incomplete
 select(where(~ n_distinct(.) > 1)) # Remove sensors with zero variance
```

```{r Verify the new features}
train_engineered %>% select(unit_id, cycle, rul, target_label, s_11_roll_avg) %>% head()
```

This engineering step is inspired by "Condition-Based Monitoring" (CBM) principles where technicians watch for trending deviations in vibration or temperature. By automating this with slide_dbl, the analyst captures the "historical behavior and evolving state" of the asset, providing the context necessary for detecting anomalies before they breach safety thresholds.

**5.0 The Analyze Phase**

**5.1 Sensor Correlation and Machine Learning Optimization:**

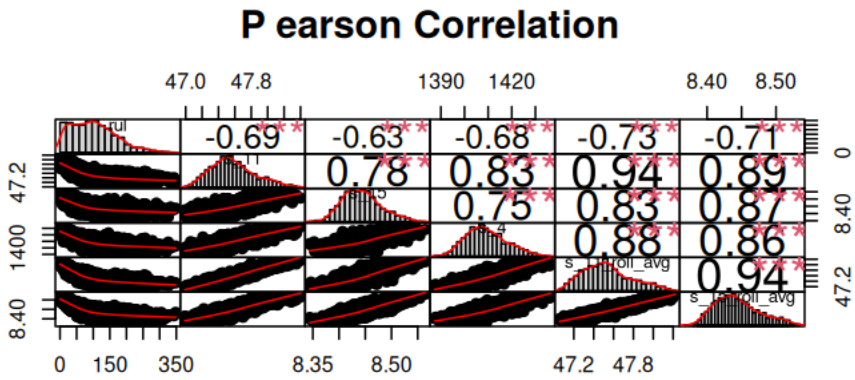
My objective is to validate the selected features and train a model capable of high-precision classification. It is pertinent to ensure that the statistical correlations match the physical realities of the engine's failure modes, such as HPC degradation.

**5.2 Validating Predictive Power with PerformanceAnalytics:**

The chart.Correlation function provides a comprehensive visualization of the correlation matrix, combining histograms, scatter plots, and correlation coefficients in a single view. This tool allows the analyst to identify which sensors—such as Sensor 11 (Ps30) or Sensor 15 (Bypass Ratio)—show the strongest relationship with the RUL.

```{r validating predictive power with performanceanalytics}
```

```
# Selecting a representative subset for correlation analysis

correlation_data <- train_engineered %>%

  select(rul, s_11, s_15, s_4, s_11_roll_avg, s_15_roll_avg) %>%

  sample_n(2000)


# Generating the correlation chart

# Top diagonal: Pearson coefficients; Bottom diagonal: Scatter plots

chart.Correlation(correlation_data, histogram = TRUE, pch = 19)
```




The visualization reveals that rolling averages (s_11_roll_avg) have higher correlation coefficients than raw values, confirming that the "temporal context" is statistically significant.

**5.3 The tidymodels Optimization Framework:**

The predictive model is built using the tidymodels ecosystem, which ensures a reproducible workflow from data splitting to performance evaluation. I will compare a Random Forest model against a Logistic Regression baseline. Random Forest is often preferred in this domain because it is "robust to outliers and unscaled features" and can capture the non-linear interactions between thermodynamic parameters.

```{r loading workflows}

library(workflows)

```

```{r model building}

set.seed(42)

# Splitting data into Training and Testing sets

pdm_split <- initial_split(train_engineered, prop = 0.8, strata = target_label)

pdm_train <- training(pdm_split)
```

```
pdm_test <- testing(pdm_split)

# Recipe definition: Normalizing and removing redundant predictors
pdm_recipe <- recipe(target_label ~., data = pdm_train) %>%
  # Keeping IDs as context variables but not predictors
  update_role(unit_id, cycle, rul, new_role = "id") %>%
  step_rm(has_role("id")) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_zv(all_predictors())

# Model Spec: Random Forest with 500 trees
rf_spec <- rand_forest() %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

# Workflow Integration
pdm_workflow <- workflow() %>%
  add_recipe(pdm_recipe) %>%
  add_model(rf_spec)

# Model Fitting
pdm_fit <- pdm_workflow %>% fit(data = pdm_train)
```

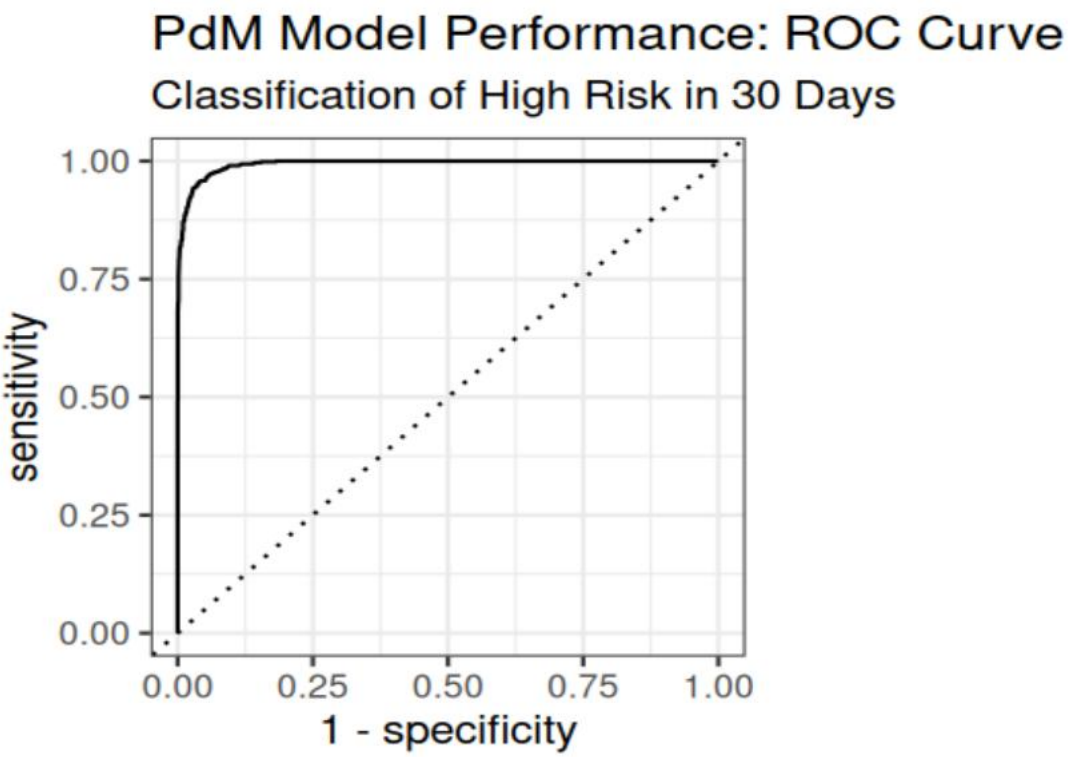
This optimization model must prioritizes **Recall** (the sensitivity to actual failures).

**5.4 Visualization:**

I will use an ROC (Receiver Operating Characteristic) curve to demonstrating the model's ability to discriminate between classes across different thresholds.

```{r Performance Visualization}
# Predicting on test data
pdm_preds <- pdm_test %>%
```

```
  select(target_label) %>%
  bind_cols(predict(pdm_fit, pdm_test)) %>%
  bind_cols(predict(pdm_fit, pdm_test, type = "prob"))

# Plotting the ROC Curve
library(tidymodels)

pdm_preds %>%
  roc_curve(truth = target_label, .pred_HighRisk) %>%
  autoplot() +
  labs(
    title = "PdM Model Performance: ROC Curve",
    subtitle = "Classification of High Risk in 30 Days"
  )
```



The plot above provides a visual assessment of the model's ability to distinguish between "High Risk" (engines expected to fail within 30 days) and "Normal" states.

Here is a detailed breakdown of the components and what they indicate about my Predictive Maintenance (PdM) model:

**i. Understanding the Axes**

- **Sensitivity (Y-axis):** This is the **True Positive Rate**. It measures the model's ability to correctly identify actual "High Risk" engines. A high sensitivity means the model is successfully catching the majority of imminent failures before they happen.
- **1 - Specificity (X-axis):** This is the **False Positive Rate**. It measures how often the model incorrectly flags a "Normal" engine as "High Risk." In a chemical plant setting, this represents "unnecessary maintenance" on a healthy machine.

**ii. Interpretation of the Curve**

- **Proximity to the Top-Left Corner:** The solid black line rises very sharply and hugs the top-left corner of the graph. This indicates **excellent model performance**. It shows that the model can achieve very high sensitivity (catching almost all failures) while maintaining a very low false positive rate (rarely calling for unnecessary repairs).
- **Area Under the Curve (AUC):** Though not explicitly labeled, the area under this solid line appears to be very high (likely above 0.95). In industrial benchmarks for the NASA C-MAPSS dataset, robust ensembles like Random Forest often reach these levels, indicating high "discriminative power".
- **The Dotted Diagonal Line:** This represents a "random classifier" (like flipping a coin). Any curve significantly above this line demonstrates that the model has learned meaningful patterns from the sensor data (such as Sensor 11 and Sensor 15 trends) rather than guessing.

**iii. Operational Significance**

From an engineering and strategic perspective, this plot confirms that my model is reliable for decision-making. Because the curve is so steep, you can choose a threshold that maximizes **Recall** (Sensitivity) to ensure no failure is missed, which is critical for avoiding the **40x cost differential** of an emergency shutdown (CAD $261,100) compared to a planned repair (CAD $6,500).

In summary, this plot demonstrates that my data processing "Engineering Edge"—using rolling averages to smooth sensor noise—has effectively translated raw thermodynamics into a high-fidelity prediction of asset failure.

**6. Conclusion:**

The transition to predictive maintenance represents a convergence of thermodynamic principles, data science rigor, and financial discipline. By following the Data Analytics framework, the industrial organization moves from the "crash" of reactive failure to the "control" of data-driven prevention.

The technical core of the project—engineered with R, slider, and tidymodels—provides a lead-time signal that allows for maintenance during lowest clinical or production utilization periods, thereby "minimizing revenue displacement". As a chemical engineer, the model ensures process safety and

environmental compliance; as a data analyst, it provides a high-fidelity window into complex system degradation; it also secures the long-term value and reliability of the investment portfolio.

Predictive maintenance is not merely about preventing a machine from breaking; it is about protecting the "productive capacity" of the entire industrial ecosystem. In the escalating downtime crisis of 2026, those who leverage the "Engineering Edge" to catch degradation early will gain the control, foresight, and resilience necessary to thrive in an increasingly volatile global market.